

Lesson 5 — Rules, Score And Feel

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

One variable, not four flags

```
let state = "MENU";
// MENU | PLAYING | PAUSED | GAME_OVER
```

Four flags = 16 combinations, 12 of them nonsense.
One state = 4 combinations, all valid. The bad states become **impossible to write**.

Use it once, in the loop

```
function update(dt) {
  if (state !== "PLAYING") return;
  updatePaddles(dt);
  updateBall(dt);
}
```

Not `if (!paused)` inside every object — you will forget one.

Sound, no files needed

```
let audio = null;
function beep(freq, duration) {
  if (!audio) audio = new AudioContext();
  const osc = audio.createOscillator();
  const vol = audio.createGain();
  osc.frequency.value = freq;
  osc.type = "square";
  vol.gain.setValueAtTime(0.08,
audio.currentTime);
  vol.gain.exponentialRampToValueAtTime(
  0.0001, audio.currentTime + duration);
  osc.connect(vol);
  vol.connect(audio.destination);
  osc.start();
  osc.stop(audio.currentTime + duration);
}
```

Create the context **lazily** — browsers block audio until the user interacts. Always **fade**, or you hear a click.

Screen shake

```
ctx.save();
if (shakeTime > 0) {
  const fade = shakeTime / shakeTotal;
  ctx.translate(
    (Math.random()-0.5)*2*strength*fade,
    (Math.random()-0.5)*2*strength*fade);
}
drawEverything();
ctx.restore(); // ALWAYS
```

Particles

```
const angle = Math.random() * Math.PI * 2;
speedX = Math.cos(angle) * speed;
speedY = Math.sin(angle) * speed;
```

Angle first → an even circle. Random x and y separately → a lumpy square.

Remove backwards:

```
for (let i = a.length - 1; i >= 0; i--) {
  if (dead) a.splice(i, 1);
}
```

Forwards + splice = skipped items.

Fade anything

`alpha = life / totalLife` → slides 1 → 0.

The 100 ms rule

Juice should last about **a tenth of a second**. Long enough to notice, short enough to never get in the way.

Section A — Recall

A1. Why is one `state` variable better than four true/false flags? Use numbers in your answer. [3 marks]

A2. What is a *transition*, and what does it mean when a key press has no transition for the current state? [2 marks]

A3. Why must `ctx.save()` always be matched with `ctx.restore()` ? What goes wrong? [2 marks]

A4. Roughly how long should a screen shake last, and why not longer? [2 marks]

A5. A particle has `life = 0.12` and started with `0.45` . What alpha should it be drawn at? Write the expression and the number. [3 marks]

Section B — Predict the output

B1. An array holds 6 particles and **all six** have run out of life. This loop runs once. How many are left, and why? [4 marks]

```
for (let i = 0; i < particles.length; i++) {  
  if (particles[i].life <= 0) {  
    particles.splice(i, 1);  
  }  
}
```

B2. What does the player see after about one second? [3 marks]

```
function render() {  
  ctx.translate(shakeX, shakeY);  
  drawEverything();  
  // no save, no restore  
}
```

B3. The game is in `GAME_OVER`. What happens when the player holds the up arrow?

[3 marks]

```
function update(dt) {  
  if (keys["ArrowUp"]) { player.y -= SPEED * dt; }  
  if (state === "PLAYING") { updateBall(dt); }  
}
```

B4. Describe the *shape* of the particle spray each version produces, and say which has more particles going diagonally.

[4 marks]

```
// version A  
const angle = Math.random() * Math.PI * 2;  
speedX = Math.cos(angle) * 200;  
speedY = Math.sin(angle) * 200;  
  
// version B  
speedX = (Math.random() - 0.5) * 400;  
speedY = (Math.random() - 0.5) * 400;
```

Section C — Find and fix the bug

C1. The game pauses correctly, but the particles keep flying around and one enemy keeps moving. The author added `if (!paused)` to the ball, the paddles and the score. Explain what is wrong with this *approach*, not just this instance.

[4 marks]

C2. No sound is ever heard. There is no error in the console. The `beep` function is correct. What is the most likely cause, and why would the browser do this deliberately?

[4 marks]

```
const audio = new AudioContext(); // at the top of the file, on page load
```

C3. Every beep ends with an audible click. What is missing?

[3 marks]

Section D — Build it

Start from your lesson 4 Pong.

- ☐ **Checkpoint 1.** Add a `state` variable with `MENU`, `PLAYING` and `GAME_OVER`. Space starts the game from the menu. Draw a title screen.
- ☐ **Checkpoint 2.** Add scores for both players. Draw them. First to 5 wins, and reaching 5 moves the game to `GAME_OVER` with a message saying who won.
- ☐ **Checkpoint 3.** Make space restart from the game-over screen. Check that the scores reset — forgetting this is extremely common.
- ☐ **Checkpoint 4.** Add **sound**: different pitches for paddle hits, wall hits and scoring. Play a rally with your eyes shut and see if you can follow what is happening.
- ☐ **Checkpoint 5.** Add **screen shake** (small on a paddle hit, bigger on a score) and **particles**. Find a shake duration that feels like impact rather than an earthquake.
- ☐ **Checkpoint 6.** Add a `PAUSED` state on P. Verify that *nothing* moves — including the particles.

THE EXPERIMENT THAT IS THE POINT OF TODAY

Once it works, play one rally with every effect switched off, then switch them on one at a time.

The code that decides who wins never changes. Write down which single effect made the biggest difference for the least code.</p>

Section E — Design challenge

NO RIGHT ANSWER

E1. Draw the state diagram for a game you actually play. Include the states most people forget: loading, settings, cutscene, "are you sure you want to quit?", level complete.

State diagram: boxes for states, arrows labelled with what causes each move

E2. Find a transition in your diagram that should **not** be allowed. What would go wrong if it were? (For example: what breaks if you could go from `PAUSED` straight to `GAME_OVER`?)

E3. Pick one thing in your game that currently gives no feedback at all. Design feedback for it in three different senses: something **seen**, something **heard**, and something **felt** through the controls.

E4. The uncomfortable one. Juice makes a game feel better without making it fairer, deeper, or more skilful. A slot machine is almost entirely juice. Where is the line between good feedback and manipulation? Does it matter whether the player can tell it is happening?

Stretch goals

1. **Hitstop.** Freeze the entire game for 40 ms on a paddle hit. Try 20, 40 and 150 ms. Freezing the game briefly makes it feel *more* responsive, which sounds like nonsense until you try it.
2. A fading trail behind the ball.
3. Make the paddle-hit pitch depend on *where* the ball struck, so the sound carries information.
4. A best-of-five match system, with a different message for winning the match than a round.
5. **An option to turn screen shake off.** Motion effects cause genuine nausea for some players. Then think about what else in your game should be optional.